

포트폴리오

지원자 서지민

CONTACT

✉ jn2xxcom@gmail.com

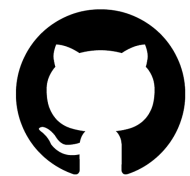
☎ 010-8549-5412

백엔드 개발부터 인프라 설계 및 운영까지, 서비스 전 과정을 직접 구축해온 엔지니어입니다.

개발	Java · SpringBoot	3개 프로젝트 백엔드 API 설계 및 구현
	Python · Django	실서비스 백엔드 유지보수 및 기능 개발
DB	MySQL · PostgreSQL	쿼리 최적화 경험, SQLD 자격 보유
클라우드 인프라	AWS	SAA 자격 보유, 3개 프로젝트 실 운영
	Terraform(IaC)	AWS 인프라 전체 코드화
CI/CD	Github Actions	ECR 연동 자동 배포 파이프라인 구축
	Bash 스크립트	배포 자동화 스크립트 작성
기타	Docker · Linux · Git	컨테이너화, 서버 운영, 브랜치 전략 허용

소개

지원자 서지민



Github
@SeoJimin1234



LinkedIn
@jimin-seo-2xx



Blog
<https://seojimin1234.github.io/>

EDUCATION

2021.03 이화여자대학교 입학
(주: 컴퓨터공학, 부: 사이버보안)

2026.08 이화여자대학교 학사 졸업 예정

EXPERIENCE

2026.03 - 현재 AWS Cloud Club(멤버)

2026.03 - 현재 연합개발동아리 IT's Time(개발자)

2025.08 - 2026.02 Frankfurt UAS(교환학생)

2025.04 - 2025.05 향해 99 데브코스 쿠버네티스(튜티)

2024.09 - 2025.02 대학생연합개발동아리 UMC(개발자)

2024.07 - 2024.08 오픈SW컨트리뷰션아카데미 ArgoCD(챌린저)

2024.02 - 2025.08 블록체인학회 EWHA-CHAIN(부학회장)

2023.09 - 2024.02 이화여대 중앙 컴퓨터동아리 ECC

2023.03 - 2024.03 지속가능발전학회 SDP(테크팀원, 홍보팀장)

2021.09 - 2024.12 문화교류연합동아리 MEDDY(홍보팀장)

SKILL

High AWS, SpringBoot, Django,
SQL, Docker

Middle Terraform, Linux, Bash

AWARD

2024.09 WEB2X ConneCTHON 대상

2024.11 이화여대 SW창업경진대회 장려상

2025.05 SW중심대학 창업경진대회 장려상

LICENSE

정보처리산업기사(2024.09 취득)

AWS SAA(2025.02 취득)

SQLD(2025.04 취득)

네트워크관리사2급(2025.07 취득)

OPIc 영어 IH(2026.03 취득)

프로젝트 목록

지원자 서지민

 Chattr 팀 작업을 위한 효율적이고 빠른 채팅 서비스	 플랫폼 Web	 담당 역할 인프라 엔지니어 (인프라 1, BE 2, FE 1)	 개발 기간 2026.04 - 2026.05
 RePLAN 실패 원인을 분석하고, 할 일을 재설계하는 To-Do List 서비스	 플랫폼 Mobile App (ios, Android)	 담당 역할 백엔드&인프라 엔지니어 (BE 2, FE 2)	 개발 기간 2026.03 - 진행 중
 TokenUs 영상을 디지털 자산으로 변환하는 NFT 기반 영상 공유 플랫폼	 플랫폼 Web	 담당 역할 백엔드&인프라 엔지니어 (BE 1, FE 2)	 개발 기간 2024.09 - 2025.11
 SASM 지속 가능한 장소를 큐레이팅하고 커뮤니티를 제공하는 서비스	 플랫폼 Mobile App, Web	 담당 역할 백엔드 엔지니어 (BE 4, FE 4)	 개발 참여 기간 2023.03 - 2024.02

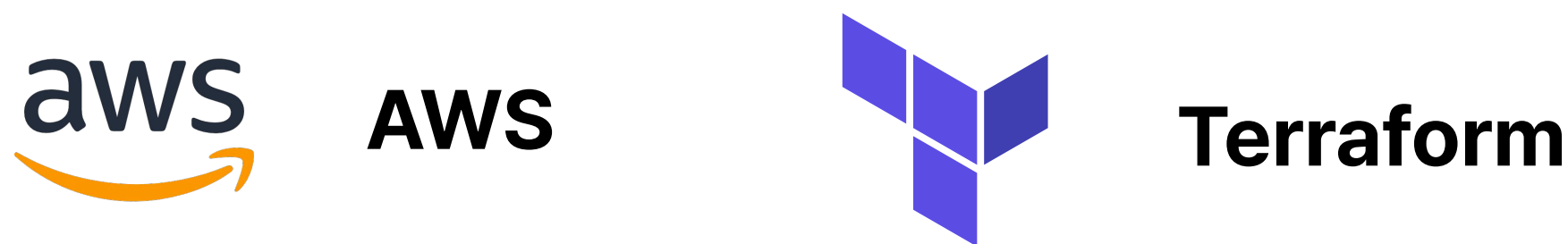


팀의 효율적인 협업을 위한 대규모 채팅 서비스

핵심 기능

워크스페이스	팀의 협업 공간. 다양한 채널 생성 가능.
채널	워크스페이스의 일부 혹은 전체 멤버로 팀 채팅
DM	1:1 채팅 서비스

기술 스택

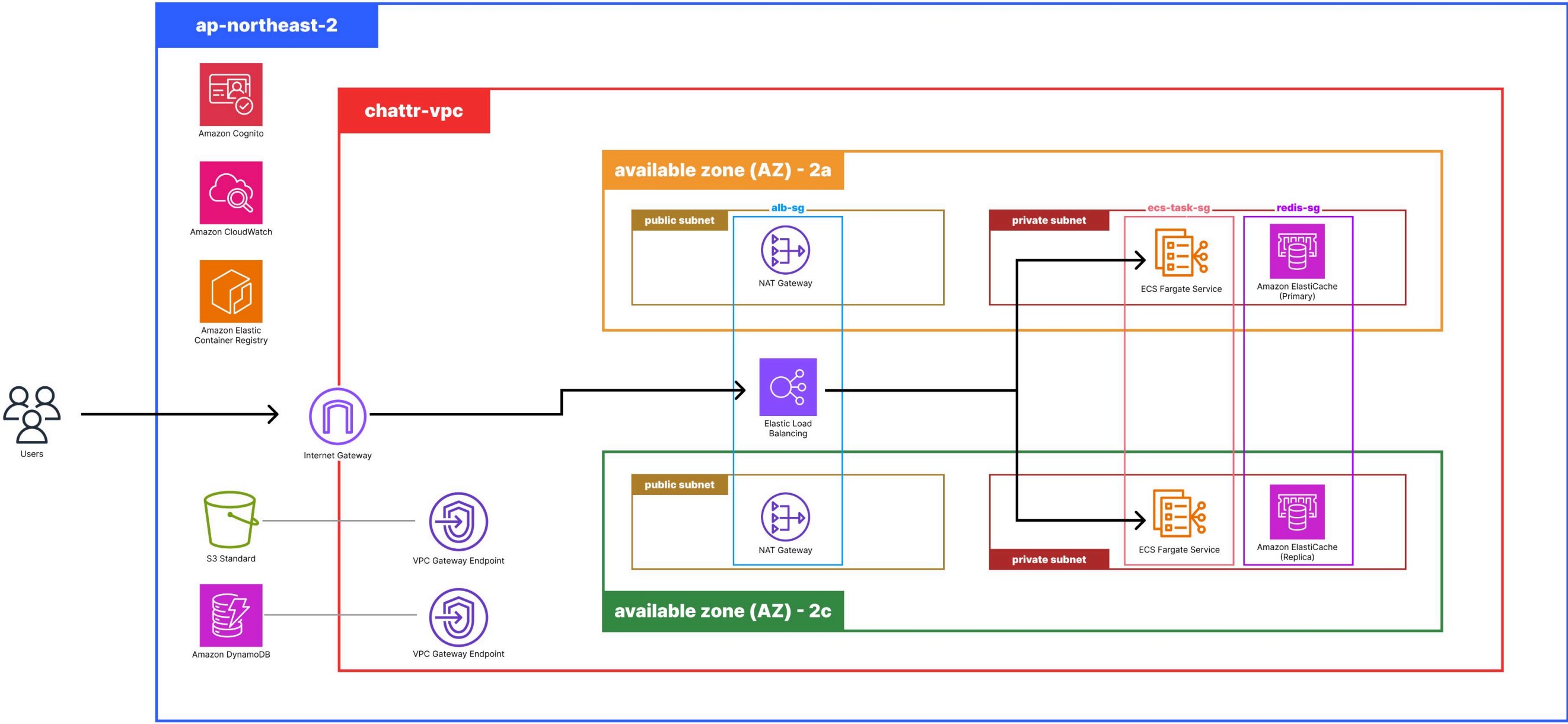


인프라 요구 조건

- 평균적으로 약 1000명의 유저가 서비스를 사용
- 최대 동시 접속 사용자 수 500명 예상
- 초당 약 50건의 메시지 전송 처리 필요
- 최대 6개월간 메시지 내역 보관 필요
- 메시지 전송 지연 시간은 2초 이내
- 채팅에서 최대 10MB 크기의 파일 첨부 기능 지원
- 멀티 디바이스(인당 최대 3대) 지원

기여

AWS 기반 클라우드 인프라 아키텍처 설계 및 구현



[클라우드 인프라 아키텍처]

보안

EC2 직접 접근 제거(Fargate), IAM 역할 분리, Cognito JWT 강제, SG 인바운드 단계적 제한, Presigned URL 단기 만료, SSM으로 시크릿 분리, Private 서브넷 배치, VPC Endpoint로 외부 노출 차단

비용 절감

Fargate 사용량 기반 과금, DynamoDB On-Demand, S3 Lifecycle 정책, VPC Endpoint로 NAT Gateway 경유 제거

성능

Redis Pub/Sub로 멀티 인스턴스 브로드캐스트, DynamoDB GSI로 쿼리 최적화, S3 직접 업로드로 ECS 부하 제거, AWS 백본 내부 통신으로 S3·DynamoDB 접근 지연 감소

가용성&안정성

Multi-AZ 서브넷 + NAT Gateway 2개, ALB 헬스체크 기반 자동 트래픽 전환, 관리형 NoSQL로 스키마 변경 유연성 확보

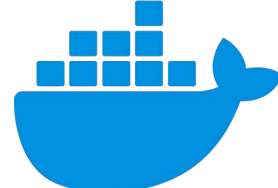
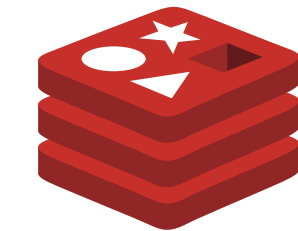
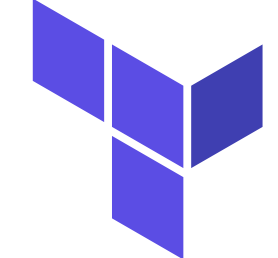
RePLAN

생성형 AI를 활용하여 **실패 원인을 분석**하고, 이를 바탕으로 할 일을 **실행 가능한 계획으로 재설계**해주는 To-Do List 서비스

핵심 기능

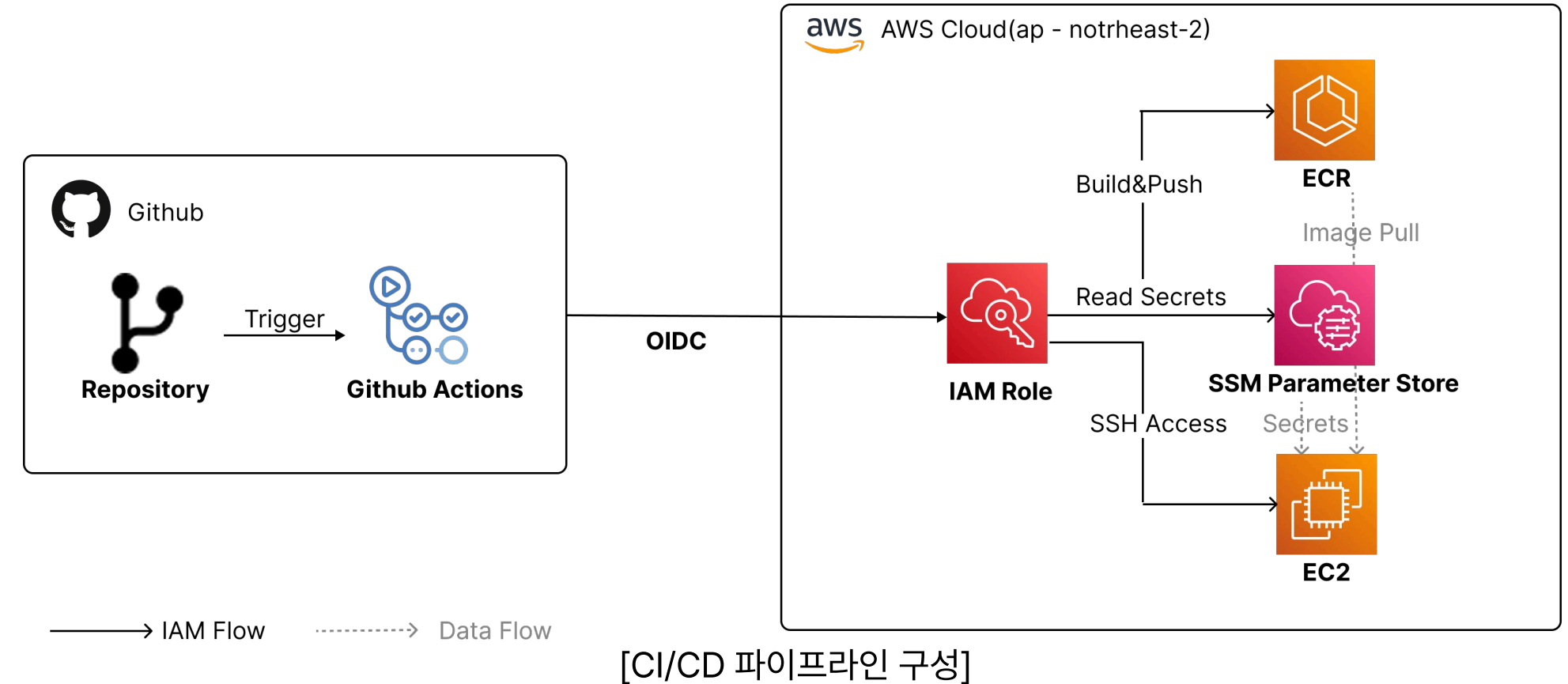
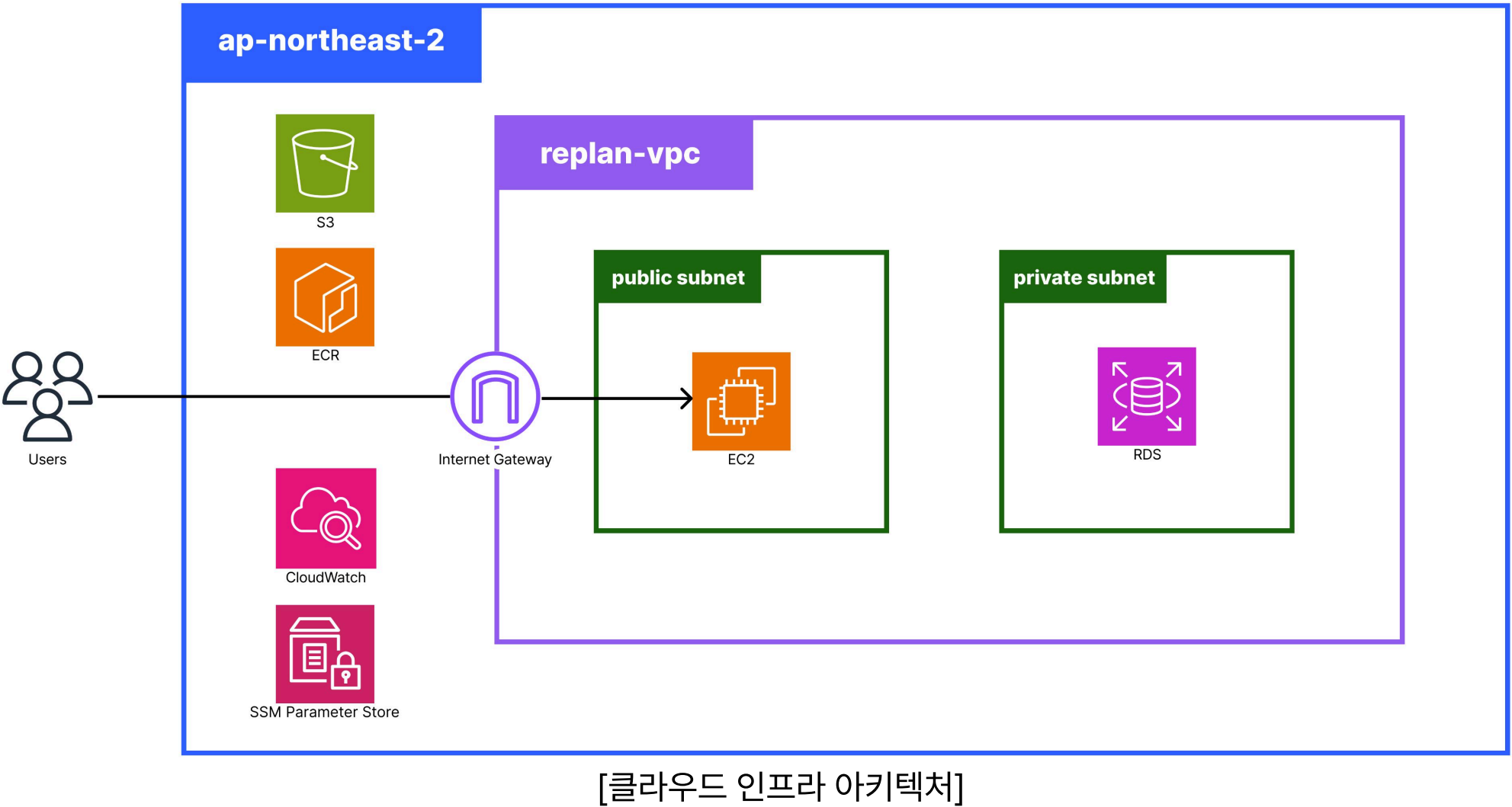
목표	사용자의 목표에 맞춘 ToDoList 추천 및 생성
홈	사용자 커스텀 ToDoList 관리
리플랜	AI 기반 실패 원인 분석 및 계획 재설계
통계 및 피드백	달성률 통계 및 심층 분석과 AI 피드백 제공

기술 스택

	AWS		Docker
	PostgreSQL		SpringBoot
	Redis		Terraform

기여

①인프라 및 CI/CD 파이프라인 설계&구현



보안 중심 설계

OIDC 기반 인증으로 고정 액세스 키를 배제하고 IAM 최소 권한 원칙을 적용. DB를 Private Subnet에 배치해 외부 접근 차단. SSM Parameter Store로 환경 변수를 분리하여 관리

컨테이너 기반 배포 자동화

Github Actions로 이미지 빌드부터 ECR 푸시, SSH 기반 앱 갱신까지 전 과정을 자동화.

운영 효율성 및 데이터 관리

Docker Compose로 3개의 컨테이너를 운영하며 배포 일관성을 확보. Redis 캐싱과 Nginx 리버스 프록시로 서비스 응답 성능을 향상. AWS RDS로 DB를 독립적으로 운영하고 Flyway로 스키마 변경 이력을 관리.

모니터링 및 관측성

CloudWatch로 인프라 리소스와 로그를 실시간 수집하여 장애 발생 시 신속한 원인 파악 및 대응이 가능한 관측성 확보

기여

②월간 통계 리포트 생성 배치 작업

병목 분석

- 유저 1명 처리 시간 → DB 집계 및 저장은 수십 ms, Gemini 호출은 수 초 단위
- Gemini 4,000 RPM → 스레드 2개로도 한도에 못 미침
- 실제 병목은 t2.micro EC2 서버 스펙

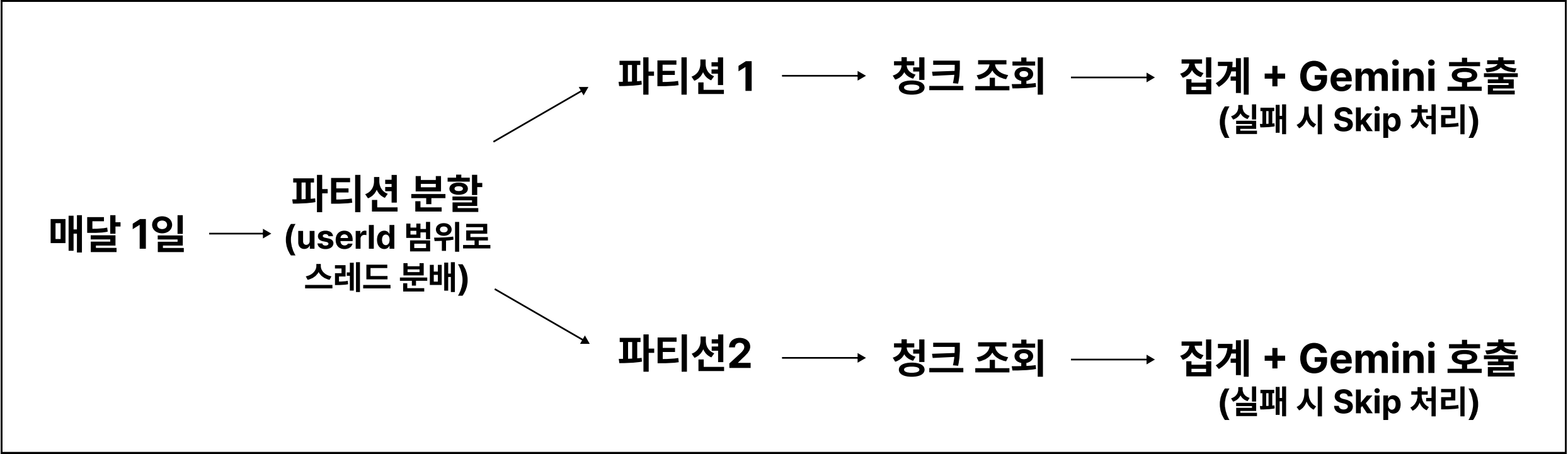
환경 제약 확인

- CPU 크레딧, HikariCP 제약 → 버스터블 인스턴스 스레드 과다 시 크레딧 고갈 위험
- RAM 1GB 힙 고갈 위험 → findAll() 전체 조회 시 JVM 힙 부족

파라미터 결정

- 파티션(스레드) 수 = 2
- 청크 사이즈 = 50명

[파라미터 결정 과정]



[배치 작업 다이어그램]

배치 프레임워크 선정

월간 통계는 이벤트성이 없는 스케줄 일괄 처리라는 점, 별도 브로커 없이 기존 DB로 Job 상태 영속화와 실패 재시작이 가능하다는 점을 근거로 Spring Batch 채택.

스펙 기반 파라미터 결정

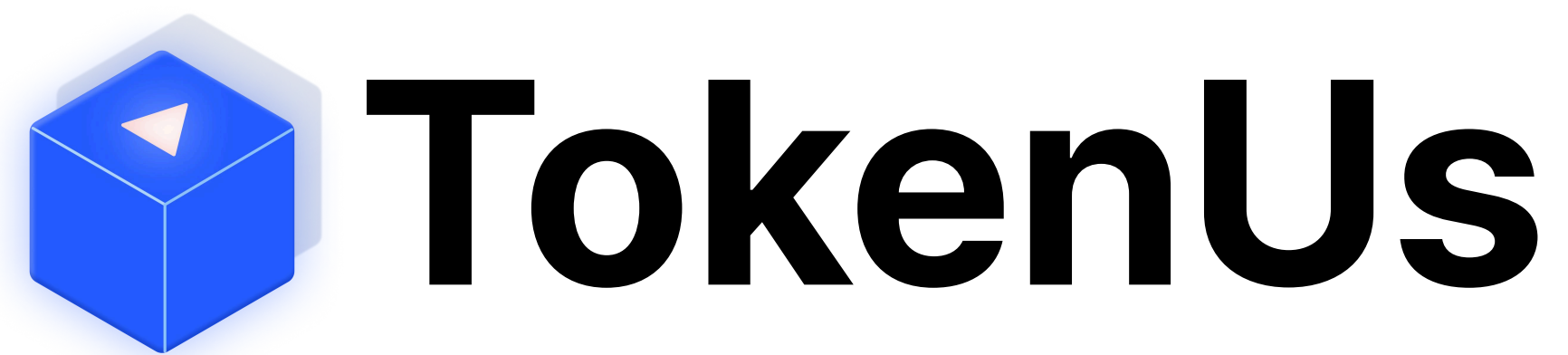
Gemini는 병목이 아니었고, 실제 제약은 t2.micro 서버 스펙임을 확인해 스레드 수와 청크 사이즈를 결정(좌측 그림 참고)

실패 격리 및 재시작 보장

Skip Policy로 유저 단위 실패를 격리해 전체 배치 중단을 방지. Job Repository가 실행 상태를 기록해 중단 시 실패 지점부터 재개 가능.

비활성 유저 조기 분기

전월 활동이 없는 유저는 Gemini 호출 없이 빈 리포트만 저장하도록 분기 처리.



무분별한 영상 무단 복제를 방지하고 원저작자의 권리를 보호하기 위한 블록체인 기반 영상 공유 플랫폼

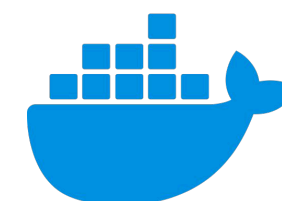
핵심 기능

- 영상 유사도 검사** 영상의 고유성을 보장→NFT 발행의 정당성 확보
- 영상 NFT 발행** 영상 콘텐츠의 NFT 발행
- 영상 NFT 거래** 소유권 이전부터 거래 이력 관리까지 전 과정을 자동화
- 영상 시청 및 반응** 영상 시청과 반응, 실시간 영상 NFT 가격 공개

기술 스택



AWS



Docker



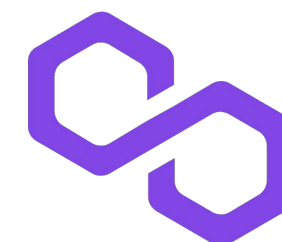
MySQL



SpringBoot



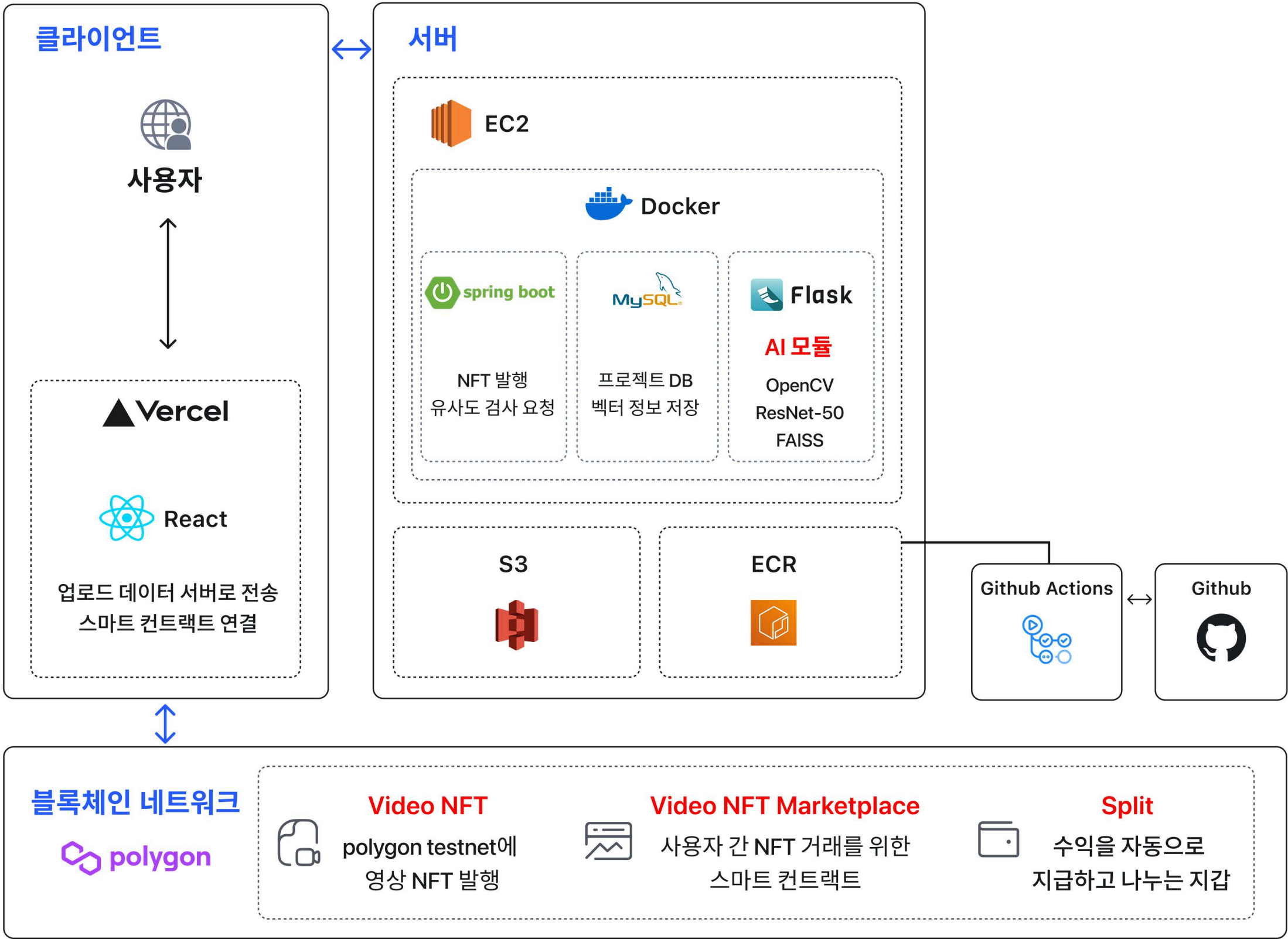
Solidity



Polygon

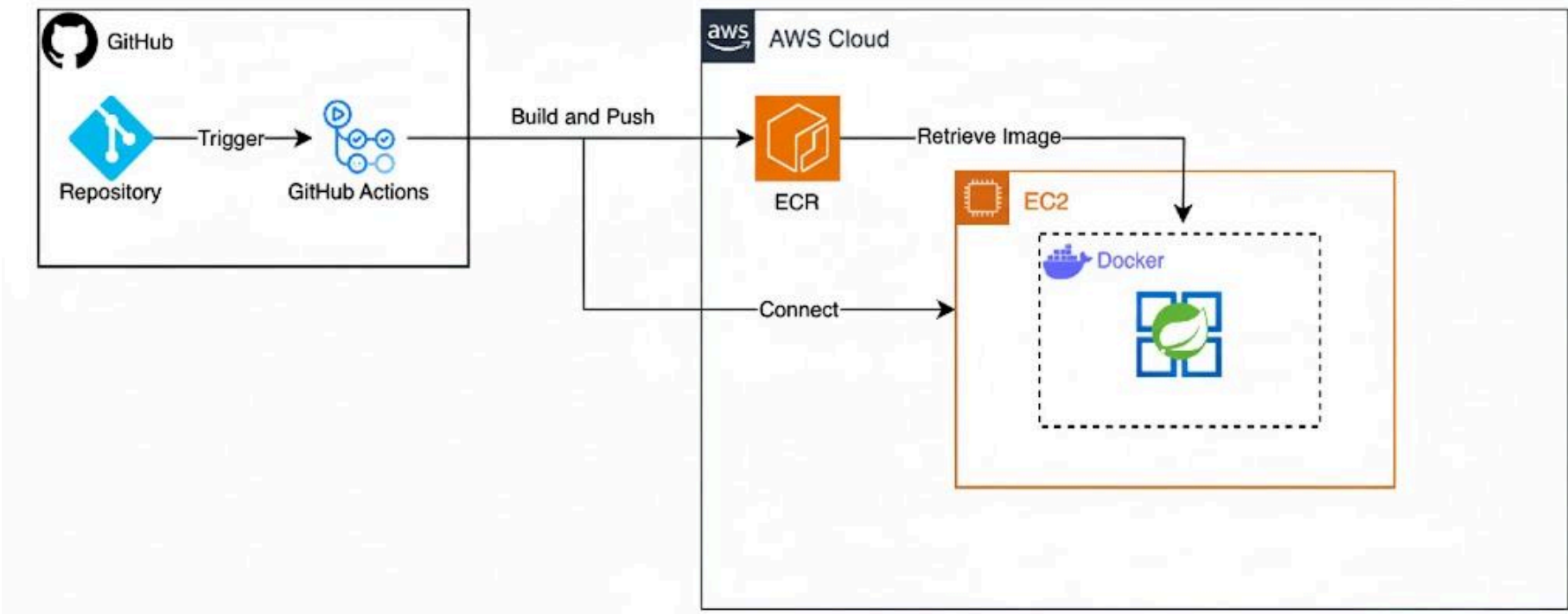
기여

① 아키텍처 설계



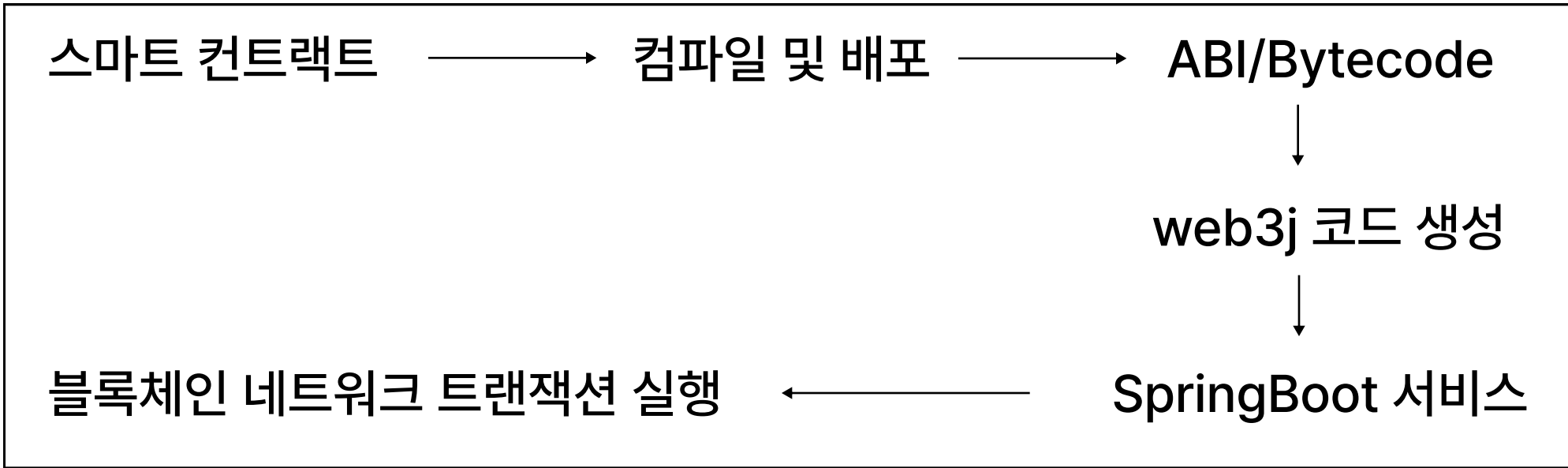
[시스템 아키텍처]

② CI/CD 파이프라인 구축, 배포 자동화



[CI/CD 파이프라인 구성]

③ 블록체인 스마트컨트랙트 연동



[연동 다이어그램]

트러블 슈팅

① ML 연산 Timeout 문제 → WebSocket 도입 및 비동기 처리를 통한 해결

문제 상황

```
유사도 결과 수신:
Object {
  elapsed_time: 48.51
  max_segment_similarity: 0.99159974
  message: "Similarity check failed"
}
```

영상 유사도 검사 처리 시간이 평균 48초 이상 소요되어, **SpringBoot API 서버의 동기요청 처리 한계를 초과**하게 됨.

결과적으로 클라이언트는 결과를 받지 못한 채 연결이 끊기는 현상이 발생.

원인 및 현상 분석

[근본 원인: ML 연산의 병목]

영상 데이터 특성상 실시간 응답을 보장하기 어려운 무거운 작업임에도, SpringBoot가 동기 방식으로 결과를 기다린 것이 근본 원인.

[현상: 자원 낭비]

클라이언트가 응답을 받기 위해 무한정 연결을 유지해야 하므로 서버 자원 효율성이 떨어지는 구조

해결

[WebSocket 도입]

HTTP 일회성 연결 대신, 클라이언트와 서버 간 지속적인 연결 통로를 구축.

[비동기 처리 및 결과 Push]

요청 즉시 "접수 완료"를 반환해 타임아웃을 방지하고, ML 연산이 완료되는 시점에 서버가 클라이언트에게 소켓을 통해 결과를 즉시 전송

인사이트

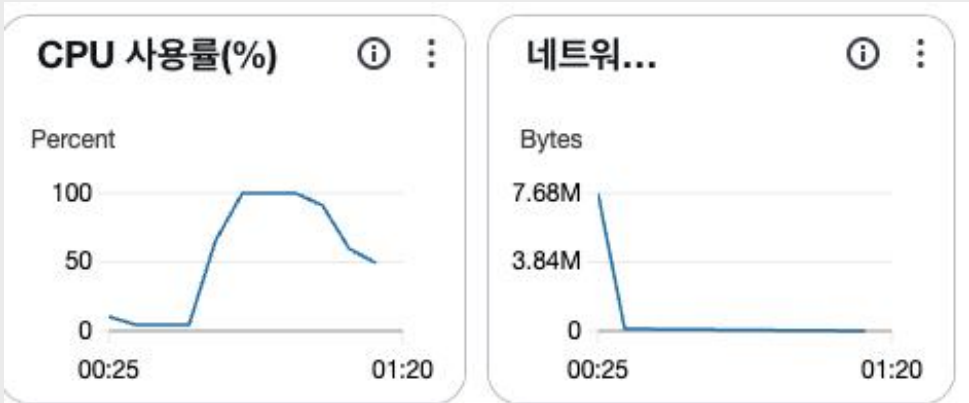
데이터 처리 시 비동기 방식과 지속적 연결의 중요성을 체감했으며, 서버 자원 효율성을 고려한 아키텍처 설계의 필요성을 배움.

다만 본 프로젝트는 ML서버 →SpringBoot→Client로 이어지는 서버 단방향 푸시이므로, 추후 SSE(Server-Sent-Events)로 전환 시 별도 의존성 없이 더 가벼운 구현이 가능할 것이라 판단.

트러블 슈팅

② 서버 불능 문제 해결(CPU 사용률 99%→50%이하)

문제 상황



EC2에 SpringBoot 컨테이너 배포 직후 **CPU 사용률이 99%를 상회**하고 메모리가 고갈되어 EC2가 정상 작동하지 않음.

동일 호스트의 MySQL, Flask 컨테이너까지 연쇄적으로 불능 상태에 빠짐.

원인 및 현상 분석

[초기 구동 부하]

SpringBoot 기동 시 의존성 로딩, JPA 초기화로 CPU 및 메모리 사용량 급증.

[리소스 고갈 및 병목]

프리티어 t2.micro(RAM 1GB, vCPU 1개)의 **제한된 환경**에서 메모리 부족 → CPU 병목 발생 및 시스템 마비

[독점 및 연쇄 마비]

호스트 **자원을 컨테이너 간 격리 없이** 공유 → MySQL, Flask 컨테이너 사용 리소스 박탈

해결

애플리케이션 배포 설정 최적화

- **JPA 설정 변경:** ddl-auto를 none으로 설정하여 서버 기동 부하 감소
- **컨테이너 자원 격리:** SpringBoot 컨테이너에 CPU 0.5/메모리 512M 제한

인스턴스 메모리 환경 개선

2GB 스왑 공간 구성→기동 시 메모리 피크를 흡수해 여유 확보

인사이트

제한된 리소스를 가진 환경에서는 **애플리케이션의 설정 최적화**뿐만 아니라 **시스템 레벨의 리소스 관리**가 필수적임을 학습.

단일 호스트에서 다용도 컨테이너를 함께 운영하는 환경에서는, 특정 컨테이너의 부하가 전체 시스템을 무너뜨리지 않도록 **컨테이너 레벨의 자원 격리와 제한이 설정**이 필요함을 깊이 이해함.



SDGs 목표 실현에 기여하는 여가 공간을 '지속가능한 공간'으로 정의하고, 이에 대한 지도화 및 큐레이팅 서비스를 제공

핵심 기능

Place(Map)	지속가능한 공간을 모아 지도화
Forest	커뮤니티의 일종으로, 유저가 생각을 자유롭게 공유
Story	특정 Place에 대한 Story 정리
MyPage	좋아요를 하거나 리뷰 작성한 콘텐츠 모아보기

기술 스택



MySQL



Django



AWS



Docker

기여

① 신규 기능 설계 및 구현

기존 시스템과 정합성을 유지하는 MyPage 기능
API 설계 및 구현

② 운영 이슈 대응 및 버그 수정

출시 전 자체적인 탐색적 테스트를 수행, 접수된 엠티케이스
버그를 파악하고 해결

③ 레거시 코드 개선

프로덕트 공통 컨벤션 및 디자인 패턴을 준수하여
기존 코드와의 통일성 유지

에러 목록

에러 작성 시 최대한 구체적으로 작성 부탁드립니다!
(사진 캡처, 발생 위치와 구체적인 현상 작성)

표

↑ 우선순위

해결: 체크 표시됨

+ 필터

우선순위

테크 메모

발생 위치

해결

Aa 이름

1순위		MyPage	✓	유저 프
1순위		Map	✓	맵에서 에 맞춰
1순위	BE 구현 완료(#544)	etc	✓	통합 검
1순위		Map	✓	리뷰에
1순위		Story	✓	스토리
1순위		MyPage	✓	유저 정
2순위		Forest	✓	Board 튼을 누 X 오류
2순위		Map	✓	홈에서 당하는 목록들

☐

#472 fix: Add SamePlaceStory informations

#481 by SeoJimin1234 was merged on Jul 30, 2023 • Approved

☐

fix: Remove duplication at mypick_forest

#487 by SeoJimin1234 was merged on Aug 1, 2023 • Approved

☐

#487 fix: Remove my_forest Duplication

#488 by SeoJimin1234 was merged on Aug 2, 2023 • Approved

☐

fix: Add Nickname in Mypage/Forest

#496 by SeoJimin1234 was merged on Aug 9, 2023 • Approved

☐

fix: Modify Story vegan_category Selector

#501 by SeoJimin1234 was merged on Aug 14, 2023 • Approved

☐

fix: Return Nickname at Curation List

#506 by SeoJimin1234 was merged on Aug 18, 2023 • Approved

☐

fix: Replace preview with Summary in Story Search

#509 by SeoJimin1234 was merged on Aug 20, 2023 • Approved

☐

#520 fix: User semi Cateory Blank True

#526 by SeoJimin1234 was merged on Sep 23, 2023 • Approved

☐

#540 fix: Curation Order set default

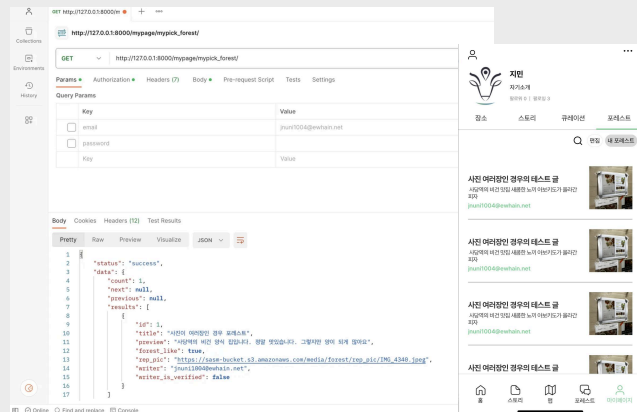
#541 by SeoJimin1234 was merged on Oct 14, 2023 • Approved

14

트러블 슈팅

객체 중복 리턴 문제 해결 → API 응답 속도 20% 개선

문제 상황



해시태그가 포함된 글의 리스트 조회 시,
포함된 해시태그 개수만큼 동일한 객체가
중복되어 리턴되는 현상 발생

데이터베이스에 객체가 여러개 생성 된
것은 아니나, 조회(GET) 결과에서만 동일
한 ID의 데이터가 반복됨

원인 및 현상 분석

[SQL Join의 특성]

ManyToMany 관계인 해시태그를 필터링 조건에 포함하면서 SQL 내부적으로 JOIN이 발생. 글 1개가 해시태그 N개를 가질 경우 DB 레벨에서 N개 행이 생성되어 QuerySet에 그대로 반영됨.

[N+1 문제 발견]

중복된 글 객체 각각에 대해 연관된 해시태그를 조회할 경우, 개수만큼 추가 쿼리가 발생하는 N+1 구조임을 발견.

해결

[distinct() - 중복 제거]

SQL 레벨에서 중복된 행을 제거. DB에서 전송되는 데이터 양과 서버의 객체 생성 부하를 감소. 데이터 정합성 확보.

[prefetch_related() - N+1 해결]

해시태그 데이터를 미리 조회하여 게시글 수만큼 발생하던 쿼리를 단 2회로 병합.

인사이트

M2M 필터링이 Join을 유발하고, 연관 객체 접근이 N+1 쿼리로 이어지는 구조를 직접 진단하며 ORM과 SQL의 동작 원리를 학습.

쿼리 병합만으로 응답 속도가 약 20% 개선되며 쿼리 횟수가 성능에 직결됨을 체감.